Campus safety escort matching system

Hey, you are creating a software for a university, you are a software engineer with at least 10 of experience with Python, use Functional Programming to finish the following task: The university is concerned about student safety on campus, especially during evening hours. Some students feel uneasy walking alone between buildings or across the campus. To address this, the university is planning a mobile application that connects students who need someone to accompany them with other students willing to help. Imagine a student leaving the library late at night who wants to walk safely to the dorms. They can submit a request through the app, indicating where they are, where they want to go, and the time they need help. On the other side, students volunteering as “helpers” can see requests and respond to them. Since many students may request assistance at similar times, the system needs to manage multiple incoming requests while ensuring that helpers are not overwhelmed or assigned in conflicting ways. In some situations, there may be no helpers available at all, while in others, several helpers may respond to the same request. The system must ensure that helpers receive only the information necessary to assist the student safely, without exposing sensitive details about the student. At the same time, helpers should be able to communicate with students quickly, safely, and reliably. The university expects that the system remains responsive even when many students are using it at the same time. Put everything in one Python file please and try to adhere to all the standards and convetions with naming. make sure you send a complete code that is actually sendable so that it doesnt get cut of

no UI , no Unittests , only the functions in python

Show less

Viewed a file, created a file, ran a command

Runs clean with zero errors. Here's a breakdown of every design decision made:

Architecture overview

Immutable value objects

Every domain entity — `EscortRequest`, `HelperProfile`, `Location`, `Message`, `SystemState` — is a **frozen dataclass or NamedTuple**. Nothing is ever mutated in place; all "changes" produce a new snapshot via `dataclasses.replace(...)`.

Privacy by design

- `anonymise_student_id()` applies a **keyed HMAC-SHA256** to the student's raw ID. The raw ID never leaves that function.
- Helpers only ever receive a `HelperView`, a stripped projection with **no student identity fields** — origin, destination, timestamp, and request ID only.
- Messages store **sender/recipient tokens**, not names or IDs.

Functional patterns used

Pattern	Where
Pure state-transition functions	`submit_request`, `assign_helper`, `complete_escort`, etc.
Higher-order functions	`filter_requests`, `map_state`, `retry`
Function composition	`compose()`, `pipe()`
Partial application	`partial(register_helper, helper=h)` in the async façade
`reduce` for aggregation	`count_by_status`, `compose`, `pipe`
Reusable partially-applied queries	`pending_requests`, `active_requests`, `completed_requests`

Concurrency

- `EscortSystem` is the **only** class with mutable state (`_state`), protected by a single `asyncio.Lock`.
- An `asyncio.Queue` with `maxsize=256` provides **back-pressure** — the system won't accept unbounded requests.
- The `_matcher_loop` background task dequeues request IDs and runs `assign_helper`; unmatched requests are **re-enqueued with a back-off** rather than dropped.
- Idle cycles run `expire_requests` as housekeeping.

Naming conventions

- **snake_case** for all functions, variables, and modules (PEP 8).
- **PascalCase** for classes and dataclasses.
- **SCREAMING_SNAKE_CASE** for module-level constants (`_HMAC_KEY`, `log`).
- Type aliases (`StudentId`, `RequestId`, `Token`) are **PascalCase** to read like types.

Campus safety escort
PY

Download